

Understanding JSON

(JavaScript Object Notation)

Lecture Objectives

- Understand the purpose of JSON in modern data exchange.
- Master the syntax rules for creating valid JSON objects.
- Learn how to organize data using nested objects and arrays.
- Practice converting data between JSON strings and usable code.
- Identify common formatting errors and basic security risks.

How Do We Move Data Between Different Languages?

- Programs use different internal methods to store and organize information.
- Direct communication requires a shared, language-neutral format.
- Data must be simplified into text to travel across networks.
- The format must be efficient for machines but readable by humans.

What is JSON?

- JSON (JavaScript Object Notation) is a lightweight, text-based format for representing structured data.
- It is designed to be completely language-independent and portable across any platform.
- A primary goal is to be human-readable, making it easy for developers to debug and understand.
- It focuses on minimal syntax to ensure fast processing and low overhead for machines.

JSON vs. XML

- JSON uses a much simpler syntax that avoids the repetitive opening and closing tags required by XML.
- The smaller file size of JSON allows for faster data transmission over the internet.
- Modern web browsers can process JSON natively as code, whereas XML requires more complex parsing.
- Data structures in JSON map directly to common programming concepts like arrays and objects.
- JSON is easier for developers to read at a glance, leading to fewer errors and faster debugging.

Anatomy of a JSON Object: The Key-Value Pair

- A JSON object is always contained within a set of curly braces to define its boundaries.
- Data is organized into pairs consisting of a unique "key" followed by a specific "value."
- The key and value are separated by a colon to establish their relationship clearly.
- Multiple pairs within the same object are separated by commas to keep the data organized.
- Keys must always be wrapped in double quotes to ensure they are interpreted as strings.

```
{  
  "name": "Kamal",  
  "age": 25,  
  "isStudent": true,  
  "city": "Colombo"  
}
```

JASON Object Rules

- Data must be strictly wrapped in double quotes as single quotes are not recognized.
- Every opening brace or bracket must have a corresponding closing partner to be valid.
- A colon must be used specifically to separate a property name from its assigned value.
- Commas are required between multiple items but are strictly forbidden after the final element.
- Number values must follow standard mathematical notation without leading zeros or special characters

JSON Data types

```
{  
  "full_name": "Saman Perera",  
  "age": 24,  
  "gpa": 3.85,  
  "is_enrolled": true,  
  "scholarship_details": null,  
  "hobbies": ["Photography", "Gaming"]  
}
```

- Text values must always be wrapped in double quotes to be interpreted as strings.
- Numeric data is written as plain digits and can be either whole integers or decimals.
- Logical states are defined using the literal keywords true or false without quotes.
- The keyword null is used to explicitly indicate that a data field has no value.
- Complex values can consist of ordered lists or even other nested JSON objects.

JASON Arrays: Handling Lists of Information

- Arrays are defined by a set of square brackets and are used to store an ordered collection of values.
- Unlike objects, arrays do not use keys and instead rely on the position of the data to organize it.
- A single array can contain a mix of different data types, including strings, numbers, and even other objects.
- Each item within the brackets must be separated by a comma to maintain a valid list structure.
- Arrays allow a single property to hold multiple related pieces of information, such as a list of tags or user IDs.

```
{  
  "name": "Kamal",  
  "subjects": ["Maths", "Science", "ICT"],  
  "marks": [85, 78, 92],  
  "isActive": true  
}
```

JSON Object Nesting: Organizing Data Inside Data

- Object nesting means placing one JSON object inside another object.
- It is used to represent more detailed and structured information.
- Nested objects help group related data together logically.
- This makes complex data easier to organize and access in programs.

```
{
  "name": "Kamal",
  "age": 25,
  "contact": {
    "email": "kamal@example.com",
    "phone": "0771234567"
  },
  "address": {
    "street": "12 Lake Road",
    "city": "Colombo",
    "country": "Sri Lanka"
  },
  "job": {
    "title": "Software Engineer",
    "company": {
      "name": "Tech Solutions",
      "location": "Colombo"
    }
  }
}
```

JSON vs. JavaScript Objects:

- JSON is a strictly text-based format, whereas JavaScript objects are actual data structures living in computer memory.
- Keys in JSON must always be wrapped in double quotes, but JavaScript allows keys to be written without quotes in most cases.
- JavaScript objects can contain functions and logic, while JSON is limited strictly to data values.
- The JSON format is universally recognized by many programming languages, but JavaScript objects are specific to their own environment

JSON (Text Format)

```
json {"name": "Kamal", "age": 25}
```

JavaScript Object

```
js const person = { name: "Kamal", age: 25 };
```

Parsing: Turning a String into an Object (JSON.parse)

- Parsing is the process of converting a JSON string into a usable object.
- In web development, this means transforming data received from APIs or servers into a live JavaScript object.
- This ensures the data can be easily accessed and used within the program.
- In JavaScript, the **JSON.parse()** method is used to perform this conversion.

```
const jsonString = '{"name":"Kamal","age":25,"city":"Colombo"}';

const person = JSON.parse(jsonString);

console.log(person.name); // Kamal
console.log(person.age);  // 25
```

Serialization

- Serialization is the process of converting a data structure (like a JavaScript object) into a **JSON string** so it can be stored or sent over a network.
- In web development, this means “freezing” an object into text format that other systems can easily understand and later rebuild.

```
const user = {  
  name: "Kamal",  
  age: 25  
};  
  
const jsonString = JSON.stringify(user);  
console.log(jsonString);
```

JSON and Modern APIs

- JSON is the standard format used to send and receive data between a browser and a server.
- In modern web apps, APIs allow the browser to request and exchange data without reloading the page.
- This makes applications faster, more dynamic, and interactive.
- The browser sends requests and receives JSON responses to update content in real time.

Linters, Formatters, and Viewers

- Linters help detect errors and improve code quality by checking JSON structure and syntax.
- Formatters automatically organize and clean up JSON to make it easier to read.
- Viewers display JSON data in a structured and user-friendly way.
- These tools make working with JSON faster, clearer, and less error-prone.